

My Own ChatGPT v2

System Introduction Report

112550051

GitHub Repository: https://github.com/yiihsinn/genai_hw1

Overview

My Own ChatGPT v2 is a **zero-dependency web AI assistant** built with **vanilla JavaScript** on the frontend and **Node.js built-in modules** on the backend. The system supports streaming chat, multimodal image input, persistent memory, automatic model routing, tool calling, multi-session local history, export, regeneration, and voice input. The backend acts as an application server so provider access, memory handling, and request validation are **centralized** instead of being exposed directly in the browser.

Frontend Design

The browser side is responsible for **interaction, short-term context, and rendering**. `app.js` coordinates prompt submission, image upload, editing, export, and session switching. `state.js` stores recent context and multiple chat branches in `localStorage`. `api.js` handles HTTP requests and SSE consumption so assistant replies can stream token by token. `memory-ui.js` and `tools-ui.js` render persistent memory cards and tool execution cards. This structure **separates UI logic from transport and local state management**.

Backend Design

The backend uses a **native Node.js HTTP server**. `server.js` initializes the application and connects the router, memory store, SSE helpers, and model service. `router.js` exposes `/api/chat`, `/api/memory`, and `/api/health`. The main orchestration logic lives in `gemini.js`, which acts as an **NVIDIA NIM proxy**. It builds OpenAI-compatible chat payloads, validates image input, performs **model auto-routing**, runs **tool-calling loops**, and forwards **streamed responses** through SSE. `tools.js` implements weather, calculator, and web-search functions.

Memory Mechanism

The system combines **short-term context** with **typed long-term memory**. Short-term memory is the recent chat history selected before each request. Long-term memory is stored in `memory.json` by `memory.js` and **survives across sessions**. To better match AI agent memory concepts, entries are typed as **episodic, semantic, or procedural**. When a conversation becomes long, the backend triggers a memory pipeline that performs **summarization, structured extraction, and reflection-style insight generation**. The results are written back to persistent storage and later retrieved as relevant context for future prompts.

Multimodal, Routing, and Tools

The system supports **text-plus-image input** using OpenAI-style multimodal message content. When an image is attached, the backend automatically switches to a **vision-capable model**. Auto-routing also selects different models for long code-heavy prompts, lightweight requests, and translation or summarization tasks. When tools are enabled, the assistant can call backend-defined functions for **weather, calculation, and web search**. Tool calls and results are streamed back as separate SSE events so the user can observe intermediate system behavior.

Summary

Overall, the system is designed to be **simple, inspectable, and practical**. The frontend manages interaction and recent context, while the backend **centralizes model access, memory processing, routing logic, and tool execution**. This modular design makes the project suitable for demonstration and

future extension, especially for coursework focused on **AI agents, long-term memory, and multimodal applications**.

System Architecture Diagram

